

LeetCode 424: Longest Repeating Character Replacement

PhonePe

Flipkart

Goldman Sachs

Problem Statement

At most **k** times you can choose any character of a string and change it to any other uppercase English letter.

Goal: Find the length of the longest substring containing the same letter after performing the operations.

Example & Logic Walkthrough

String: A A D Y A R A S | **k = 2**

If we replace: D → A and Y → A

Resulting Substring: A A A A A R A S

Answer: **5**

Key Observations & Variables:

- Let the current window length be **len**.
- Let the maximum occurring character frequency in the current window be **maxfreq**.

- The total number of replacements needed to make all characters in the window identical is $len - maxfreq$.
- **Validity Condition:** A window is only valid if $len - maxfreq \leq k$ because otherwise, we don't have enough allowed operations to support the replacements.

Approach 1: Brute Force

For every possible substring, find the maximum frequency of any character and compute the replacements required. If the substring is valid, update the maximum answer.

```
int ans = 0;
for(int i = 0; i < n; i++) {
    vector<int> freq(26, 0);
    for(int j = i; j < n; j++) {
        freq[s[j] - 'A']++;

        int maxfreq = 0;
        for(int c = 0; c < 26; c++) {
            maxfreq = max(maxfreq, freq[c]);
        }

        int len = j - i + 1;
        int replacements = len - maxfreq;

        if(replacements <= k) {
            ans = max(ans, len);
        }
    }
}
return ans;
```

Time Complexity: $O(26 \times N^2)$

Space Complexity: $O(1)$

Approach 2: Optimal Sliding Window

This approach uses the sliding window technique to dynamically adjust the window bounds. We keep expanding the right pointer (r) and maintain the count of character frequencies. If the window becomes invalid, we shrink it from the left (l) until it becomes valid again.

```
class Solution {
public:
    int characterReplacement(string s, int k) {
        int n = s.length();
        int l = 0;
        int r = 0;
        int ans = 0;
        vector<int> freq(26, 0);
        int maxfreq = 1;

        while(r < n) {
            freq[s[r] - 'A']++;
            maxfreq = max(maxfreq, freq[s[r] - 'A']);

            while((r - l + 1) - maxfreq > k) {
                freq[s[l] - 'A']--;
                l++;
            }

            ans = max(ans, r - l + 1);
            r++;
        }

        return ans;
    }
};
```

Time Complexity: $O(N)$

Space Complexity: $O(1)$